

LightNobel 與 FPGA Triangle Multiplication Kernel : 技術摘要

本文整理三部分內容：（1）LightNobel（Han、Choi、Kim，ISCA 2025，arXiv 2505.05893）的運作方式，這是一個針對蛋白質結構預測模型（PPM）的軟硬體協同設計加速器；（2）我們針對 Triangle Multiplication einsum 提出的最佳化步驟（v0~v3），以 spatial accelerator 為對象；（3）用來評估它們的分析方法。標示（我們的解讀）的內容，論文中沒有明確寫出。

1. 背景：ESMFold 與 Pair Representation

1.1 整體流程

LightNobel 的基準模型是 ESMFold。它從長度為 L 的胺基酸序列預測 3D 結構，分為三個階段：

階段	功能	主要資料
Input embedding	ESM-2（3B）蛋白質語言模型	sequence representation $s : L \times 1024$
Folding trunk	48 個 Protein Folding Block，最多 4 次 recycling	s 與 pair representation $z : L \times L \times 128$
Structure module	預測原子座標；distogram head 由 z 產生	3D 結構

z 的每個元素 $z[i][j]$ （pair representation 的一個 *token*）是一個 128 channel 的向量，描述胺基酸 i 與 j 之間的關係。每個 block 以 residual 的方式更新狀態（ $z \leftarrow z + \Delta z$ ），依序為：sequence attention、sequence transition、sequence-to-pair projection、Triangle Multiplication（outgoing）、Triangle Multiplication（incoming）、Triangle Attention（starting node）、Triangle Attention（ending node）、pair transition。

1.2 Triangle Multiplication

以 outgoing 版本為例，channel 維度 $C = 128$ ，更新如下：

```
z_ln = LayerNorm_in(z)
a     = mask ⊙ sigmoid(W_ag · z_ln) ⊙ (W_ap · z_ln)           (逐 token, 128 → 128)
b     = mask ⊙ sigmoid(W_bg · z_ln) ⊙ (W_bp · z_ln)
x[i][j][c] = Σ_k a[i][k][c] · b[j][k][c]                     (einsum "ikc,jkc->ijc")
g     = sigmoid(W_g · z_ln)
Δz    = g ⊙ (W_z · LayerNorm_out(x))
```

除了 einsum，所有運算都是 token-local 的：每個 token 只用到自己的 128 個值和權重。einsum 是唯一跨 token 的運算。對每個 channel c ，它是一個 $L \times L$ 的矩陣乘法 $x_c = a_c \cdot b_c^T$ ；128 個 channel 彼此獨立（batched matrix multiplication）。incoming 版本計算 $x[i][j][c] = \sum_k a[k][i][c] \cdot b[k][j][c]$ ，即 $a_c^T \cdot b_c$ 。

每個輸出 token 中，einsum 需要 $2 \cdot L \cdot C$ 次運算（ $L = 1,000$ 時為 256,000 次），六個 128×128 的 linear 層需要 $6 \cdot 2 \cdot C^2 = 196,608$ 次。因此 L 增大時，einsum 會成為 TriMul 運算量的主體。

一次 Triangle Multiplication 中各類運算（以每個輸出 token 計）：

運算	類型	每個 token 的運算量	資料相依性
LayerNorm_in、 LayerNorm_out	正規化 (對 128 個 channel 做歸約)	$O(C)$	token-local
6 個 linear 層 (128×128)	矩陣-向量乘法	$6 \cdot 2C^2 = 196,608$	token-local, 共用權重
sigmoid、gating、mask	逐元素運算	$O(C)$	token-local
einsum	batched 矩陣乘法	$2 \cdot L \cdot C$	讀取 a 的第 i 列與 b 的第 j 列

1.3 Triangle Attention 與成長規模

Triangle Attention 沿著 z 的列 (或欄) 做 attention。它的 attention logits 每個 head 的形狀是 $L \times L \times L$ ，因此隨 L 立方成長；其他 pair representation 的張量則是平方成長。以 starting-node 版本為例， $H = 4$ 個 head、每個 head 維度 32：

```

x      = LayerNorm(z)
q, k, v = W_q x, W_k x, W_v x                (逐 token, 分成 H 個 head)
β[h][j][k] = (W_β x)[j][k][h]                (pair bias)
s[i][h][j][k] = q[i][j][h] · k[i][k][h] / √32 + β[h][j][k]
o[i][j][h] = Σ_k softmax_k(s[i][h][j][k]) · v[i][k][h]

```

ending-node 版本對 z 的轉置做相同的計算。

張量	形狀	成長
Sequence representation	$L \times 1024$	$O(L)$
Pair representation 與 TriMul 中間值	$L \times L \times 128$	$O(L^2)$
Triangle Attention logits (每個 head)	$L \times L \times L$	$O(L^3)$

LightNobel 論文指出， $L = 2,034$ 時 activation 已經是權重的 24.15 倍，需要 144 GB，超過單張 GPU 的容量。論文在 H100、不分塊 (no chunking) 下的執行時間分布 (Fig. 3) 顯示了瓶頸的轉移：

佔總執行時間	短蛋白質 (R0271)	長蛋白質 (T1269)
Protein Folding Block	83.8%	94.5%
Pair representation 資料流	69.4%	91.9%
Triangle Multiplication	36.1%	14.5%
Triangle Attention	29.0%	75.9%

2. LightNobel

2.1 概述

LightNobel 只加速 Protein Folding Block；ESM-2 與 structure module 不在加速器上執行。設計建立在四個觀察上：

1. 序列很長時，執行時間集中在 pair representation 資料流，其中又以 Triangle Attention 最重。
2. 記憶體用量主要來自 activation，而非權重。
3. 同一個 token 的各 channel 數值差異小，但 token 之間差異大，且 outlier 集中在特定 token（與 distogram 的模式相關）。因此以 token 為單位量化比以 channel 為單位更合適。
4. block 中不同位置的 activation 統計特性不同（例如 residual 路徑上 LayerNorm 之前的平均絕對值約 82，其他位置約 4）。

主要貢獻有三：軟體層面的 Token-wise Adaptive Activation Quantization (AAQ)；硬體層面的 Reconfigurable Matrix Processing Unit (RMPU)、Versatile Vector Processing Unit (VVP) 與 Token Aligner；以及不存放大中型間值（包括 L^3 的 attention logits）的 token-wise 資料流。

2.2 Token-wise Adaptive Activation Quantization (AAQ)

每個 token t 以自己的 scale 做對稱量化：

$$\begin{aligned} s_t &= \max_c |x_t[c]| / (2^{(b-1)} - 1) & (b = 4 \text{ 或 } 8) \\ q_t[c] &= \text{round}(x_t[c] / s_t) \\ \hat{x}_t[c] &= s_t \cdot q_t[c] \end{aligned}$$

outlier 是每個 token 中絕對值最大的 k 個值。它們在執行時以 top- k 運算挑出，以 INT16 保留原始精度；其餘的值 (inlier) 則被量化。activation 依其在 block 中的位置分為三組：

組別	對應的 activation	平均絕對值	每個 token 平均 outlier 數	格式
A	LayerNorm 之前、residual 路徑上	82.14	2.31	INT8 inlier + 4 個 INT16 outlier
B	LayerNorm 之後、linear 層之前	4.05	1.69	INT4 inlier + 4 個 INT16 outlier
C	其餘 activation（例如 linear 的輸出）	3.85	0.64	INT4，不處理 outlier

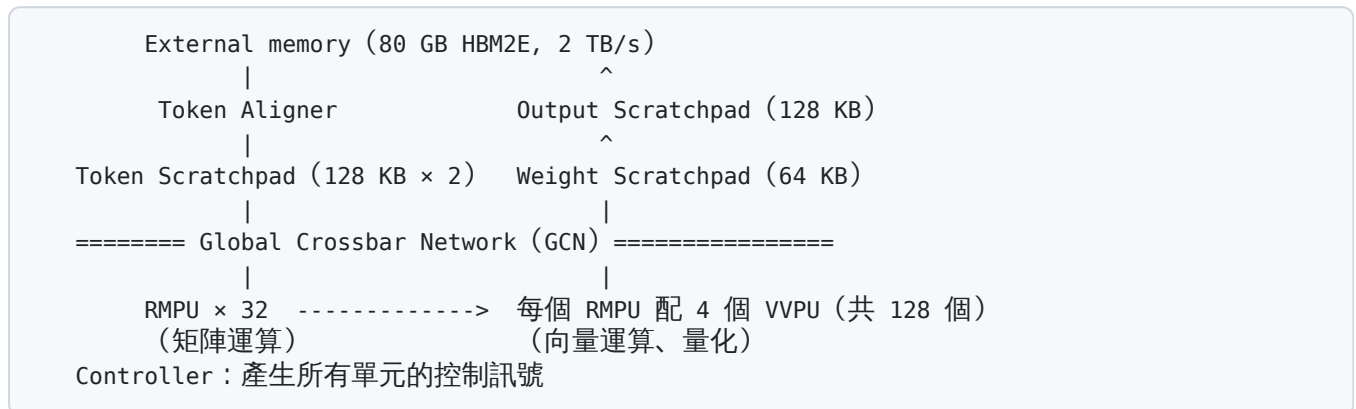
其他設計要點：

- 權重不量化，以 INT16 儲存。基準 (ESMFold) 的權重與 activation 皆為 FP16。
- 採用對稱量化，因為在處理 outlier 的前提下，其 RMSE 只比非對稱量化高 9.76%（不處理 outlier 時高 27.35%），而且硬體成本較低。

- 各組設定透過設計空間探索（精度 $\times$ outlier 數）選出，以 TM-score 與資料大小評估。未採用的選項包括：A 組用 INT4（需 32 個以上 outlier 才能維持精度）、A 組 outlier 少於 4 個（TM-score 下降）、B 組用 INT8（精度足夠但資料較大），以及 channel-wise 或 tensor-wise 粒度。
- 每個 token 的儲存格式依序為：inlier、outlier、scale、outlier 位置。多個 token 打包成 block，block 大小依記憶體通道頻寬決定；各組壓縮後的 token 長度不同。

在 Triangle Multiplication 中， z 屬於 A 組， z_{ln} 屬於 B 組， a 、 b 屬於 C 組（INT4）。

2.3 架構與執行模型



這些單元是分時共用的，而不是針對某一層焊死的固定管線：

- RMPU 執行所有矩陣類運算：linear 層、TriMul 的 einsum、 $Q \cdot K^T$ 與 attention-V。
- VVPU 執行逐元素與向量運算：LayerNorm、sigmoid、gating、residual 相加、softmax，以及執行時量化。
- GCN 讓資料以任意順序在單元之間傳遞（例如 VVPU $\rightarrow$ RMPU $\rightarrow$ VVPU）。

模型的一層由數趟（pass）計算組成：

1. 讀入：Token Aligner 從 HBM 讀入一批打包好的 token，解碼後以一個 token 一行的方式寫入 scratchpad。Token Scratchpad 採 double buffering，處理目前這批的同時載入下一批。
2. 權重：這一趟用到的權重固定放在 Weight Scratchpad。
3. 運算：VVPU 先做需要的前處理（例如 LayerNorm），RMPU 一次對最多 20 個 token 做矩陣運算，結果直接以管線送進 VVPU，做後處理與執行時量化。
4. 寫回：量化後的 token 經 Output Scratchpad 寫回。

延遲由最慢的管線階段（RMPU、VVPU 或記憶體）決定。

採用 weight-stationary 資料流，是因為一個 128×128 的 INT16 權重矩陣只有 32 KB，卻會被最多 L^2 個 token 重複使用。

2.4 RMPU：位元拆解的多精度矩陣單元

如果 INT4、INT8、INT16 各配一套專用乘法器，運算元精度混用時，用不到的那幾套就會閒置。RMPU 改為只用一種小乘法器（沿用 Bit Fusion 的做法），把數字拆成 4-bit 的小塊，再組合出較大的乘積：

$$A = \sum_i a_i \cdot 2^{(4i)}, \quad B = \sum_j b_j \cdot 2^{(4j)}$$

$$A \cdot B = \sum_i \sum_j (a_i \cdot b_j) \cdot 2^{(4(i+j))}$$

移位不需要乘法器。INT4 × INT16 需要 4 次小塊乘法，INT8 × INT16 需要 8 次，INT16 × INT16 需要 16 次。

在二補數表示中，只有最高位的小塊帶正負號，較低的小塊都是無號數。Reconfigurable Data Aligner (RDA) 對最高位小塊做 sign extension、其他小塊補 0，都擴充成 5 bit。擴充後所有小塊都落在同一種 5-bit 有號格式內（-16 ~ 15），因此乘法器的輸入為 5-bit 有號數，只需一種乘法器就能處理所有小塊。

階層結構：

層級	組成	乘法器數
PE	16 個乘法器 + 移位器 + 16-to-1 加法樹	16
PE Lane	8 個 PE	128
PE Cluster	20 個 PE Lane + Dynamic Accumulation Logic (DAL)	2,560
RMPU Engine	4 個 PE Cluster (80 個 lane、640 個 PE)	10,240

一個 C 組 token（128 個 INT4）與 INT16 權重做 128 維內積，需要 $128 \times 4 = 512$ 個乘法器（4 個 lane）；B 組 token（124 個 INT4 + 4 個 INT16）需要 $124 \times 4 + 4 \times 16 = 560$ 個（5 個 lane）。一個 cluster 放 20 個 lane，即 4 與 5 的最小公倍數，兩種情況都能完全用滿（一次 5 個或 4 個 token）。

DAL 負責處理 scale。inlier 的部分和要乘上 token 的 scale，outlier（INT16 原值）的部分和則不乘。4 個 lane 的 token，把各 lane 的和相加後乘一次 scale；5 個 lane 的 token，先把 4 個 inlier lane 相加並乘 scale，再加上 outlier lane。Arbiter 與可重組加法樹在 4-to-1 與 5-to-1 兩種歸約方式之間切換。

加法樹的多個層級都可以取出結果：

- 2 個 PE 的和：attention 中維度 32 的 head
- 4 或 5 個 lane 的和：量化後的 linear 層
- 8 或 16 個 lane 的和：未量化的運算元

2.5 VVPU：向量單元與執行時量化

每個 VVPU 包含：

- 128 條 SIMD lane：每條對應 token 的一個 channel，各有一個 16-bit ALU、本地 scratchpad 與兩級指數查表。
- Scalar Support Unit (SSU)：負責歸約與格式處理。
- Local Crossbar Network (LCN)：讓 lane 之間交換資料。

top-k 選取使用 bitonic 排序網路。它由固定連線的 compare-and-swap 元件組成，控制與資料無關；128 個值需要 $7 \cdot 8 / 2 = 28$ 級、每級 64 個比較器，求 top-k 時可以提早結束。排序時會追蹤原始位置以記錄 outlier 的位置； $k = 1$ 時得到最大值，用於 softmax 與計算 scale。

執行時量化分四步：（1）以 top-k 挑出 outlier 與 scale；（2）縮放並四捨五入 inlier；（3）由 LCN 重排成儲存格式；（4）由 SSU 做最後的對齊。完整精度的值從不寫回外部記憶體。

論文的设计空間探索 (Fig. 12) 顯示，延遲在每個 RMPU 配 4 個 VVPU、以及 32 個 RMPU 時達到飽和。超過這兩點後，新增的單元不在關鍵路徑上，或記憶體系統已無法供應更多資料。

2.6 Token-wise multi-head attention

為了不存放 L^3 的 logits，LightNobel 以類似 FlashAttention 的方式逐 token 計算 attention。每個 query 的分數分批處理，並維護執行中的統計量：

```
m_new = max(m, max(s))
ℓ      = ℓ · exp(m - m_new) + ∑ exp(s - m_new)
o      = o · exp(m - m_new) + ∑ exp(s - m_new) · V
m      = m_new
output = o / ℓ
```

硬體上的執行方式：

1. RMPU 對各 head 平行計算 $Q \cdot K^T$ 。
2. VVPU 做反量化與累加。
3. 一批的 softmax 與下一批的 $Q \cdot K^T$ 重疊執行。
4. 最後乘上 V ，寫回結果。

由於 hidden 維度很小（128），晶片內可同時容納許多 token，使 token-wise 做法在 PPM 上特別有效率。

2.7 Triangle Multiplication 的對應方式（我們的解讀）

論文沒有列出每個運算的排程。依照運算間的相依關係與論文描述的硬體能力，TriMul 分三趟執行：

趟次	範圍	運算
1	token-local，對每個 (i, k)	讀 $z \rightarrow$ VVPU LayerNorm $\rightarrow$ RMPU 四個 linear 層 $\rightarrow$ VVPU sigmoid、gating、mask、INT4 量化 $\rightarrow$ 寫出 a 、 b
2	跨 token	讀 a 、 $b \rightarrow$ RMPU einsum $\rightarrow$ VVPU 反量化與累加
3	token-local，對每個 (i, j)	VVPU LayerNorm_out $\rightarrow$ RMPU w_z 、 $w_g \rightarrow$ VVPU sigmoid、gating、residual、INT8 + outlier 量化 $\rightarrow$ 寫出 z

第 1、2 趟之間， a 、 b 必須寫回記憶體，因為 einsum 要跨 token 讀取整列資料。

2.8 評估方法與結果

論文使用以下工具評估：

- Python cycle-accurate 模擬器，與 RTL 模擬交叉驗證（平均誤差 3.30%，全部在 5% 以內）。
- SystemVerilog RTL，以 Synopsys Design Compiler 在 28 nm、1 GHz 合成。

- CACTI 7.0 與記憶體編譯器估算 SRAM，換算到 28 nm。
- Ramulator 模擬 80 GB HBM2E、2 TB/s。

工作負載為 ESMFold (ESM-2 3B)，資料集為 CAMEO、CASP14、CASP15、CASP16，比較對象為 NVIDIA A100 與 H100。

指標	結果
精度	TM-score 變化 < 0.001
加速	最多 8.44× (A100)、8.41× (H100)；與分塊的 GPU 相比為 3.85~8.44× 與 3.67~8.41×
能源效率	最多 37.29× (A100)、43.35× (H100)
峰值記憶體	比不分塊的 GPU 最多低 120.05×；比分塊的 GPU 低 1.26~5.05×
最長序列	80 GB 內可處理 9,945 個胺基酸 (CASP16 最長蛋白質 6,879 的 1.45 倍)
面積／功耗	178.80 mm ² 、67.8 W

面積與功耗 (Table 2) 以資料搬移為主。crossbar (128 個 LCN + GCN，約 125.7 mm²) 佔 70.28% 的面積與 67.95% 的功耗，RMPU Engine 只佔 18.20% 的面積。單一 VVPU 中，LCN (0.785 mm²) 約是 128 條 SIMD lane (0.115 mm²) 的 7 倍。

記憶體節省來自兩個不同的機制：

- 量化：資料流不變時，總記憶體從 121.39 GB 降到 73.50 GB (1.65 倍；T1169，3,364 個胺基酸，Table 1)。
- 不存放中間值：其中又以移除 L³ 的 attention logits 為主，這是相對不分塊的 GPU 能降低 120 倍峰值記憶體的主要原因。

pair representation 仍隨 L² 成長。

2.9 相關做法

做法	代表工作	效果	限制
GPU 系統最佳化	FastFold、ScaleFold	訓練與推論更快	記憶體仍隨 L ² 成長
分塊 (chunking)	AlphaFold／ESMFold 的 chunk 選項、AutoChunk	峰值記憶體降低	變慢；資料重複讀取
模型量化	MEFold (權重)、PTQ4Protein (per-tensor INT8)	資料變小	只量化權重對 activation 幫助有限；低精度時準確度下降
LLM 量化	SmoothQuant、LLM.int8()、AWQ	分離 outlier	inlier 只有單一精度；GPU 上 activation 量化常反而變慢
融合的 GPU kernel	OpenFold3 融合 TriMul	中間值不寫回記憶體	需手寫 kernel，受限於 GPU 架構
專用加速器	LightNobel	量化、專用單元與 token-wise 資料流	以模擬評估，未實際流片

3. Triangle Multiplication einsum 的最佳化方案 (v0 ~ v3)

本章以與平台無關的方式描述各步最佳化，並用以下參數表示：

符號	意義
L	序列長度
C	每個 token 的 channel 數 (128)
e	每個元素的 bytes (FP32 為 4)
T_I、T_J	輸出 tile 在 i、j 方向的大小
W	每個傳輸 cycle 送達的元素數 (介面寬度 / e)
P	每個 cycle 的乘加 (MAC) 數
α	每次記憶體請求的存取延遲 (cycles)
δ	乘加資料路徑的 pipeline 深度 (cycles)

3.1 問題定義與資料排列

kernel 計算 $z[i][j][c] = \sum_k a[i][k][c] \cdot b[j][k][c]$ ， $i, j < L, c < C$ ，也就是 C 個獨立的 $L \times L$ 矩陣乘法 $z_c = a_c \cdot b_c^T$ 。總工作量为 $L^2 \cdot C$ 個輸出，每個需要 L 次乘加，共 $2 \cdot L^3 \cdot C$ 次運算。三個張量都採用模型原生的 [row][col][channel] 排列。同一個 token 的 C 個 channel 是連續的 (FP32 為 $C \cdot e = 512$ bytes)，因此 a 的第 i 列與 b 的第 j 列都是 L 個連續的 token。einsum 的兩個運算元都沿列讀取，不需要轉置任何一方。

3.2 v0：基準

每個輸出元素 $z[i][j][c]$ 各自獨立計算，對 k 的歸約放在最內層。每次乘加都從外部記憶體取一個 a 元素與一個 b 元素，晶片內除了一個累加器外不保留任何資料。

- 流量： $2 \cdot L^3 \cdot C$ 次元素讀取，沒有重用。每個 a 元素對每個 j 都重讀一次，每個 b 元素對每個 i 都重讀一次。
- 存取模式：相鄰的 k 迭代存取的位址相差 C 個元素，請求無法合併成 burst，每個元素都要付出完整的延遲 α 。
- 算術強度：每 2e bytes 做 2 次運算，FP32 為 0.25 FLOP/byte。

3.3 v1：Loop tiling 與晶片內 buffer

想法：把輸出空間切成 $T_I \times T_J$ 的 tile，一個 tile 的累加器 ($T_I \cdot T_J \cdot C$ 個值) 在整個歸約過程中都留在晶片內 (output-stationary)，並把對 k 的歸約移到 tile 內部。對每個 k：

1. 把 T_I 個 token $a[i_0 \dots i_0+T_I-1][k]$ 與 T_J 個 token $b[j_0 \dots j_0+T_J-1][k]$ 載入晶片內 buffer。

2. 從這些 buffer 算出全部 $T_I \cdot T_J \cdot C$ 個乘積。

每個載入的 a 元素供 T_J 個輸出使用，每個 b 元素供 T_I 個輸出使用。

重用：晶片外讀取量從 $2 \cdot L^3 \cdot C$ 降為 $L^3 \cdot C \cdot (1/T_I + 1/T_J)$ 。 8×8 的 tile 讓讀取量降為 $1/8$ 。

連續存取：每次載入一個完整的 token，也就是 C 個連續元素，因此一個 token 可以用一次 burst 取回。

延遲 α 變成每個 token 付一次，而不是每個元素付一次。

每個 k 的成本（每 cycle 傳輸一個元素、做一次乘加）：

$$\begin{aligned} t_k(v1) &\approx (T_I + T_J) \cdot C + T_I \cdot T_J \cdot C + \text{延遲項} \\ &= 2,048 + 8,192 = 10,240 \text{ cycles} \quad (T_I = T_J = 8, C = 128) \end{aligned}$$

計算佔了 80%，所以下一步必須提高乘加速率，而不只是提高傳輸速率。

3.4 v2：寬位元傳輸與向量化

想法：einsum 的 C 個 channel 彼此獨立（不對 c 做歸約），在記憶體中也是連續的。因此不需要重新排列資料，就能每次傳輸處理 W 個 channel、每個乘加 cycle 處理 P 個 channel。v2 一致地加寬三個部分：

1. 傳輸寬度：每次記憶體傳輸 W 個元素（例如 512-bit 傳輸可攜帶 $W = 16$ 個 FP32）。一個 token 只需 C / W 次傳輸。
2. 晶片內 buffer：buffer 沿 channel 維度切成 $W (= P)$ 個獨立的 bank，每個 cycle 可以讀出 P 個運算元。
3. 資料路徑： P 個平行的乘加單元，每個 cycle 處理同一組 (i, j, k) 的 P 個 channel。

a 與 b 經由各自獨立的輸入串流讀取，因此兩者的 tile 載入可以同時進行。

每個 k 的成本：

$$\begin{aligned} t_{\text{load}} &= \max(T_I, T_J) \cdot C / W + \alpha \\ t_{\text{mac}} &= T_I \cdot T_J \cdot C / P + \delta \\ t_k(v2) &= t_{\text{load}} + t_{\text{mac}} \\ \text{例 } (T &= 8, C = 128, W = P = 16) : t_{\text{load}} = 64 + \alpha, t_{\text{mac}} = 512 + \delta \end{aligned}$$

相對於 v1，傳輸項縮小 W 倍，計算項縮小 P 倍。以 $\alpha \approx 80$ 、 $\delta \approx 15$ cycles（示意值）計算， t_k 從約 10,240 降到約 671 cycles，約 15 倍。沒有達到 16 倍，是因為 α 與 δ 不會隨寬度縮小。

3.5 v3：Double buffering

想法：v2 在載入一段 tile 資料時，乘加單元是閒置的（每 t_k 個 cycle 中有 t_{load} 個）。v3 讓 $k+1$ 的載入與 k 的計算重疊：

- 兩組 buffer：配置兩組輸入 buffer（第 0 組與第 1 組）。
- 排程：歸約開始前先把 $k = 0$ 載入第 0 組。 k 為偶數時，把 $k+1$ 載入第 1 組，同時使用第 0 組計算； k 為奇數時角色互換。
- 互不相依：載入與計算存取的 buffer 互不重疊，因此可以作為兩個獨立的硬體單元同時執行。

- 為什麼要兩組：只用一組 buffer 會產生 write-after-read hazard：下一次載入會覆蓋乘加單元仍在讀取的資料。
- 代價：輸入 buffer 的記憶體加倍；流量、傳輸寬度與乘加單元數不變。

```
v2:  [load k=0][ mac k=0 ][load k=1][ mac k=1 ][load k=2][ mac k=2 ] ...
v3:  [load k=0][ mac k=0 ][ mac k=1 ][ mac k=2 ][ mac k=3 ] ...
      [load k=1] [load k=2] [load k=3]
```

每個 tile 的成本：

```
v2:  L · (t_load + t_mac)
v3:  t_load + L · max(t_load, t_mac)
加速比 (L 很大時) = (t_load + t_mac) / max(t_load, t_mac) ≤ 2
例 (t_load ≈ 144, t_mac ≈ 527) : 671 / 527 ≈ 1.27
```

效益的上限取決於載入所佔的時間比例（例子中約 21%），符合 Amdahl's law；只有在 $t_{\text{load}} \approx t_{\text{mac}}$ 時才會達到最大值 2。

3.6 v0 ~ v3 總整理

版本	想法	晶片外讀取量（元素數）	每 cycle 乘加數	每個歸約步的時間
v0	基準，歸約在最內層	$2 L^3 C$	1	每個元素受延遲限制
v1	tiling、晶片內累加器與 buffer、以 token 為單位連續載入	$L^3 C (1/T_I + 1/T_J)$	1	$(T_I + T_J) C + T_I T_J C$
v2	寬位元傳輸 (W)、沿 channel 向量化乘加 (P)	同 v1	P	$t_{\text{load}} + t_{\text{mac}}$
v3	double buffering	同 v1	P	$\max(t_{\text{load}}, t_{\text{mac}})$

4. 計算方法

4.1 執行時間模型

不重疊： $T \approx T_{\text{compute}} + T_{\text{memory}}$
重疊： $T \approx \max(T_{\text{compute}}, T_{\text{memory}})$
 $T_{\text{compute}} = \text{運算次數} / (\text{每 cycle 運算數} \times f_{\text{clk}})$
 $T_{\text{memory}} = \text{搬運量} / \text{頻寬} + \alpha \times \text{未被重疊的請求次數}$

每種最佳化作用在其中一項：

手法	作用的項	版本
Tiling	搬運量 ↓	v1
連續 (burst) 存取	未被重疊的請求 ↓	v1
寬位元傳輸	頻寬 ↑	v2
平行乘加單元	每 cycle 運算數 ↑	v2
Double buffering	相加 → 取最大值	v3

4.2 晶片外流量

v0: $R_0 = 2 \cdot L^3 \cdot C$
v1: $R_1 = (L/T_I)(L/T_J) \cdot L \cdot (T_I + T_J) \cdot C = L^3 \cdot C \cdot (1/T_I + 1/T_J)$
 $R_0 / R_1 = 2 / (1/T_I + 1/T_J) = 8 (8 \times 8)、16 (16 \times 16)、32 (32 \times 32)$

輸出的寫入量 ($L^2 \cdot C$) 在各版本都相同， L 很大時相對於讀取量可以忽略。

4.3 算術強度與 roofline

$AI = \text{FLOPs} / \text{bytes} = 2 L^3 C / (e \cdot R)$
v0: $AI = 1 / e = 0.25 \text{ FLOP/byte (FP32)}$
v1~v3: $AI = 2 T_I T_J / (e (T_I + T_J)) = 2 (8 \times 8)、4 (16 \times 16)、8 (32 \times 32), e = 4$
可達效能 = $\min(\text{峰值運算能力}, AI \times \text{頻寬})$

Compute-bound 的條件：向量化設計的峰值運算能力為 $2 \cdot P \cdot f_{\text{clk}}$ ，兩個輸入串流提供 $2 \cdot W \cdot e \cdot f_{\text{clk}}$ bytes/s。當以下條件成立時，kernel 為 compute-bound：

$$2 P f / AI \leq 2 W e f \quad \Leftrightarrow \quad P / AI \leq W \cdot e$$

$P = W = 16$ 、 $e = 4$ 、 $AI = 2$ 時，左邊為 8、右邊為 64，因此 tiling 後的 kernel 是 compute-bound，不需要為了頻寬再加大 tile。v0 ($AI = 0.25$ ，無 burst) 則同時受延遲與記憶體限制。

4.4 Cycle 模型與校正方法

```
cycles_total = (L/T_I)(L/T_J) · cycles_tile
cycles_tile  = t_init + L · t_k + t_store
t_load       = max(T_I, T_J) · C/W + α
t_mac        = T_I · T_J · C/P + δ
t_k          = t_load + t_mac                (v2)
t_k          = max(t_load, t_mac)           (v3, 每個 tile 另加一次初始 t_load)
t_init       = t_store ≈ T_I · T_J · C/P + σ
```

結構項由迴圈邊界直接推得。常數 α 、 δ 、 σ 取決於實作平台，先用一個參考設定（合成估計或實測）校正，再用校正時沒用過的設定（其他 tile 大小、v3）驗證，目標誤差在 10% 以內。此模型沒有考慮多個串流共用同一記憶體通道時的競爭。

4.5 晶片內記憶體

```
累加器    = T_I · T_J · C · e          (8×8 : 32 KB ; 16×16 : 128 KB ; 32×32 : 512 KB)
輸入 buffer = (T_I + T_J) · C · e      (8×8 : 8 KB ; double buffering 時加倍)
```

算術強度隨 tile 大小線性成長，累加器用量則以平方成長。tile 大小選在「cycle 數對晶片內記憶體」Pareto 前緣的膝點。

4.6 容量模型

記憶體需求隨 L^2 成長，因此最大序列長度只隨可用容量 M 的平方根成長：

```
記憶體 =  $L^2$  · (每個 pair 位置的 bytes)
L_max  = sqrt( $M$  / 每個 pair 位置的 bytes)
```

相對提升與 M 無關。以 FP16 為基準（ESMFold 與 LightNobel 比較時使用的精度）：

配置	每個 L^2 位置的 bytes	$L_{\max}$ 相對值
未融合的 TriMul（約 7 份同時存在的 $L \times L \times 128$ 張量，FP16）	$7 \times 256 = 1,792$	1.00×
融合 producer/consumer，FP16 z ，INT8 a 、 b （per-token scale）	$2 \times 256 + 2 \times 132 = 776$	$\text{sqrt}(1,792 / 776) \approx 1.52\times$
同上，改用 INT4 a 、 b	$2 \times 256 + 2 \times 68 = 648$	$\text{sqrt}(1,792 / 648) \approx 1.66\times$

INT8 token 佔 128 B 加 4 bytes 的 scale，INT4 token 佔 64 B 加 4 bytes 的 scale。融合配置的假設如下：

- a 、 b 逐 token 產生（LayerNorm、linear、gating 與量化融合在一起）。
- g 在 consumer 中重算而不儲存。
- 只有 z 、輸出、 a 、 b 會寫到記憶體。

由於 L 只隨記憶體減少倍數的平方根成長，每個位置的 bytes 減少 2.3 倍， L 約提升 1.5 倍。